Update

Update的分类

首先，我们将可以触发更新的方法所隶属的组件分类：

1. `ReactDOM.render` —— `HostRoot`
2. `this.setState` —— `ClassComponent`
3. `this.forceUpdate` —— `ClassComponent`
4. `useState` —— `FunctionComponent`
5. `useReducer` —— `FunctionComponent`

可以看到，一共三种组件（`HostRoot` | `ClassComponent` | `FunctionComponent`）可以触发更新。

由于不同类型组件工作方式不同，所以存在两种不同结构的 `Update` ，其中 `ClassComponent` 与 `HostRoot` 共用一套 `Update` 结构， `FunctionComponent` 单独使用一种 `Update` 结构。

虽然他们的结构不同，但是他们工作机制与工作流程大体相同

触发更新

应用初始化

```
export function updateContainer(
  element: ReactNodeList,
  container: OpaqueRoot,
  parentComponent: ?React$Component<any, any>,
  callback: ?Function,
): Lane {
  const current = container.current;
  const eventTime = requestEventTime();
  const lane = requestUpdateLane(current); // 根据当前时间，创建一个update优先级
  const update = createUpdate(eventTime, lane); // lane被用于创建update对象
  update.payload = { element };
  enqueueUpdate(current, update);
  scheduleUpdateOnFiber(current, lane, eventTime);
  return lane;
}
```

函数组件

```
function dispatchAction(fiber, queue, action) {
  // 省略一些不相关代码
  var eventTime = requestEventTime();
  var lane = requestUpdateLane(fiber);
  var update = {
    lane: lane,
    action: action,
    eagerReducer: null,
    eagerState: null,
    next: null,
  }; // Append the update to the end of the list.
  var pending = queue.pending;
  // 省略一些不相关代码
  scheduleUpdateOnFiber(fiber, lane, eventTime);
}
```

类组件

```
const classComponentUpdater = {
  isMounted,
  enqueueSetState(inst, payload, callback) {
    const fiber = getInstance(inst);
    const eventTime = requestEventTime(); // 根据当前时间，创建一个update优先级
    const lane = requestUpdateLane(fiber); // lane被用于创建update对象
    const update = createUpdate(eventTime, lane);
    update.payload = payload;
    enqueueUpdate(fiber, update);
    scheduleUpdateOnFiber(fiber, lane, eventTime);
  },
};
```

Update对象

`ClassComponent` 与 `HostRoot` （即 `rootFiber.tag` 对应类型）共用同一种 `Update结构` 。

```
function createUpdate(eventTime, lane) {
  var update = {
```

```
    // 任务时间，通过performance.now()获取的毫秒数。
    // 由于该字段在未来会重构，当前我们不需要理解他 react18中仍然在
    eventTime: eventTime,
    // 优先级
    lane: lane,
    // 更新的类型，包括UpdateState | ReplaceState | ForceUpdate | CaptureUpdate
    tag: UpdateState,
    // 更新挂载的数据，不同类型组件挂载的数据不同。
    // 对于ClassComponent，payload为this.setState的第一个传参。
    // 对于HostRoot，payload为ReactDOM.render的第一个传参。
    payload: null,
    callback: null,
    next: null,
  };
  return update;
}
```

`FunctionComponent` 的 `Update结构`

```
var update = {
  lane: lane,
  action: action,
  eagerReducer: null,
  eagerState: null,
  next: null,
};
```

> 什么情况下一个Fiber节点会存在多个Update?

你可能疑惑为什么一个 `Fiber节点` 会存在多个 `Update` 。这其实是很常见的情况。

```
onClick() {
  this.setState({ a: 1 })
  this.setState({ b: 2 })
}
```

在一个 `ClassComponent` 中触发 `this.onClick` 方法，方法内部调用了两次 `this.setState` 。这会在该 `fiber` 中产生两个 `Update` 。

UpdateQueue 对象

在 `fiber` 对象中有一个属性 `fiber.updateQueue` , 是一个链式队列(即使用链表实现的队列存储结构)

`updateQueue` 有三种类型，其中针对 `HostComponent` 的类型我们在completeWork说过。

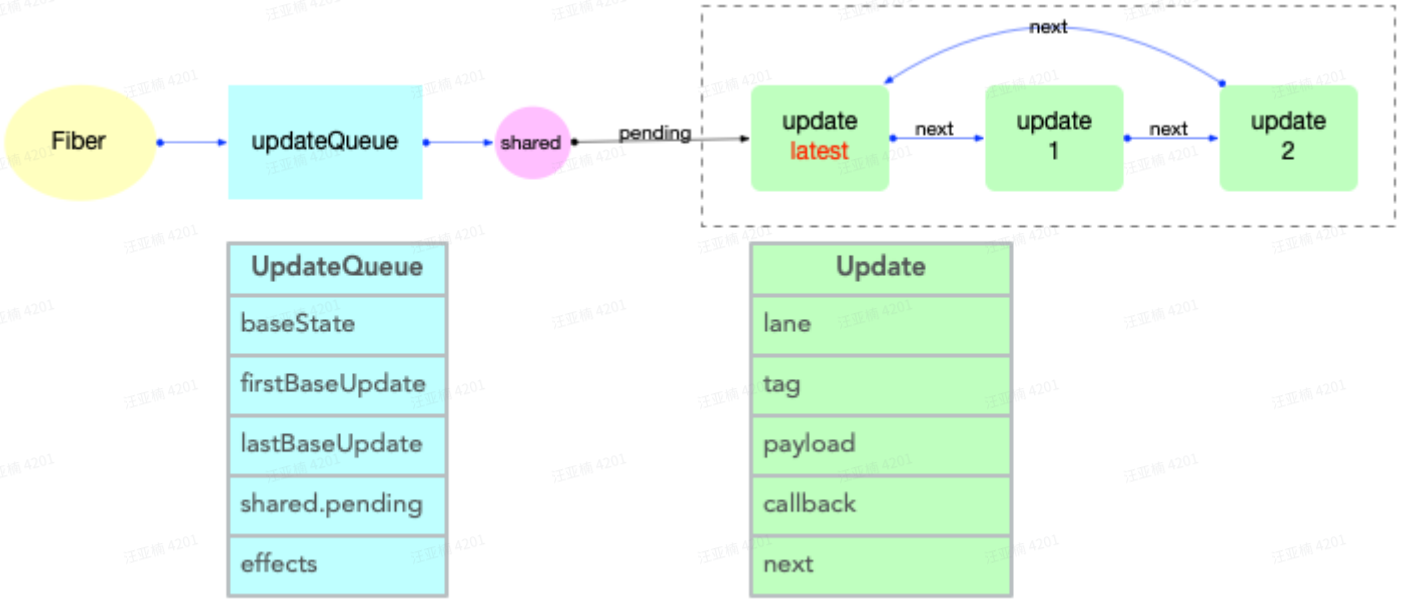
`ClassComponent` 与 `HostRoot` 使用的 `UpdateQueue` 结构

```
export type Update<State> = {};

// =============== UpdateQueue ==============
type SharedQueue<State> = {
  pending: Update<State> | null,
};

export type UpdateQueue<State> = {
  baseState: State,
  firstBaseUpdate: Update<State> | null,
  lastBaseUpdate: Update<State> | null,
  shared: SharedQueue<State>,
  effects: Array<Update<State>> | null,
};

```

属性解释:

1. `UpdateQueue`
   - `baseState` : 表示此队列的基础 state
   - `firstBaseUpdate` : 指向基础队列的队首
   - `lastBaseUpdate` : 指向基础队列的队尾
   - `shared` : 共享队列
   - `effects` : 用于保存有 `callback` 回调函数的 update 对象, 在 `commit` 之后, 会依次调用这里的回调函数.
2. `SharedQueue`
   - `pending` : 指向即将输入的 `update` 队列. 在 `class` 组件中调用 `setState()` 之后, 会将新的 update 对象添加到这个队列中来.

`updateQueue` 是 `fiber` 对象的一个属性, 所以不能脱离 `fiber` 存在


Fiber
updateQueue
shared
pending
next
update
latest
next
update
1
next
update
2
UpdateQueue
baseState
firstBaseUpdate
lastBaseUpdate
shared.pending
effects
Update
lane
tag
payload
callback
next